

# Python Assignment 1

## Aleksandr Chernysh

### Q1 The E-Commerce Cart Analyzer

The Scenario: You are analyzing a customer's shopping cart. The cart is represented as a list of dictionaries, where each dictionary holds the item's name, category, and price. You need to calculate the total checkout price, applying a specific discount, and figure out which unique departments the user shopped in.

Your Task:

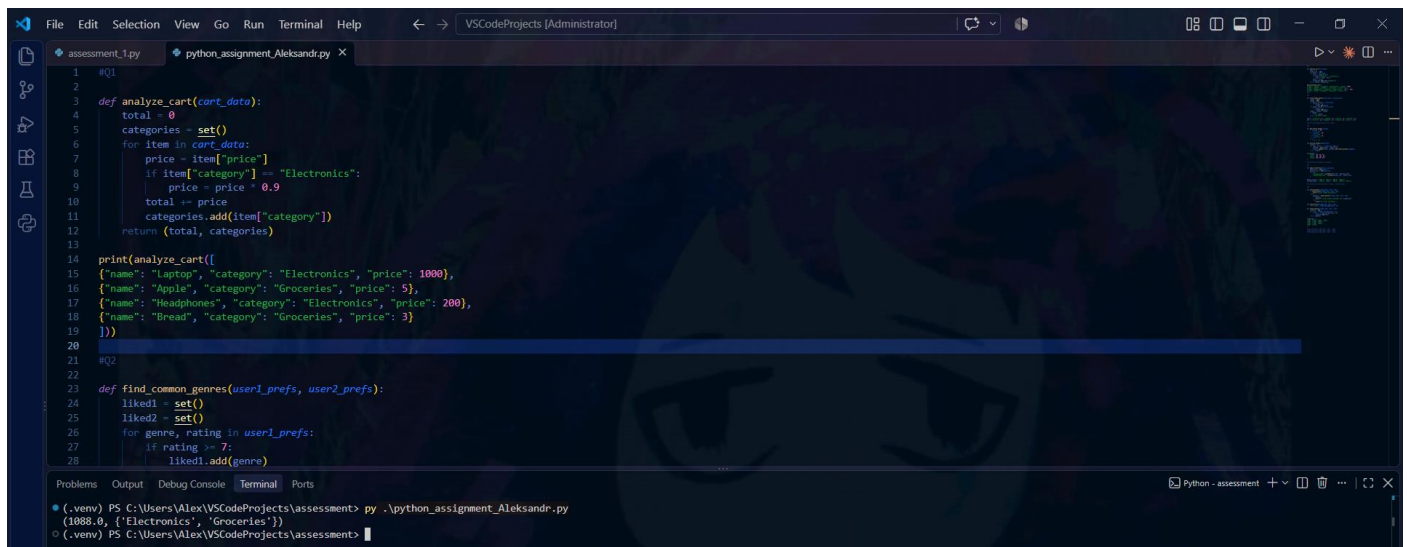
1. Write a function called `analyze_cart(cart_data)`.
2. Loop through the list of dictionaries.
3. If an item's category is "Electronics", apply a 10% discount to its price. Otherwise, the price remains normal.
4. Keep a running total of the final prices.
5. Extract all the categories into a unique collection (no duplicates allowed).
6. Return a Tuple containing: (final\_total\_price, collection\_of\_unique\_categories).

Code:

```
def analyze_cart(cart_data):
    total = 0
    categories = set()
    for item in cart_data:
        price = item["price"]
        if item["category"] == "Electronics":
            price = price * 0.9
        total += price
        categories.add(item["category"])
    return (total, categories)

print(analyze_cart([
{"name": "Laptop", "category": "Electronics", "price": 1000},
{"name": "Apple", "category": "Groceries", "price": 5},
{"name": "Headphones", "category": "Electronics", "price": 200},
{"name": "Bread", "category": "Groceries", "price": 3}
]))
```

Output:



The screenshot shows a VS Code editor with a Python file named `python_assignment_Aleksandr.py`. The code defines a function `analyze_cart` that takes a list of dictionaries as input. It iterates through the list, applying a 10% discount to items in the "Electronics" category and calculating the total price. It also collects unique categories into a set. The function returns a tuple containing the total price and the set of categories. The code is then executed, and the output is displayed in the terminal window at the bottom. The output shows the total price as 1088.0 and the unique categories as {'Electronics', 'Groceries'}.

```
def analyze_cart(cart_data):
    total = 0
    categories = set()
    for item in cart_data:
        price = item["price"]
        if item["category"] == "Electronics":
            price = price * 0.9
        total += price
        categories.add(item["category"])
    return (total, categories)

print(analyze_cart([
{"name": "Laptop", "category": "Electronics", "price": 1000},
{"name": "Apple", "category": "Groceries", "price": 5},
{"name": "Headphones", "category": "Electronics", "price": 200},
{"name": "Bread", "category": "Groceries", "price": 3}
]))
```

```
(.venv) PS C:\Users\Alex\VSCodeProjects\assessment> py .\python_assignment_Aleksandr.py
(1088.0, {'Electronics', 'Groceries'})
(.venv) PS C:\Users\Alex\VSCodeProjects\assessment>
```

## Question 2: The Movie Matchmaker

The Scenario: Two friends want to watch a movie together, but they have different tastes. You are given their preferences as lists of tuples, where each tuple contains a (Genre, Rating out of 10). You need to find which highly-rated genres they have in common.

Your Task:

1. Write a function called `find_common_genres(user1_prefs, user2_prefs)`.
2. Loop through each user's preferences.
3. If a user rates a genre 7 or higher, consider it a "Liked" genre.
4. Isolate the "Liked" genres for both users into two separate Sets.
5. Use set operations to find the genres that appear in both users' "Liked" sets.
6. Return the resulting set of common genres. If there are none, return "No match found".

Code:

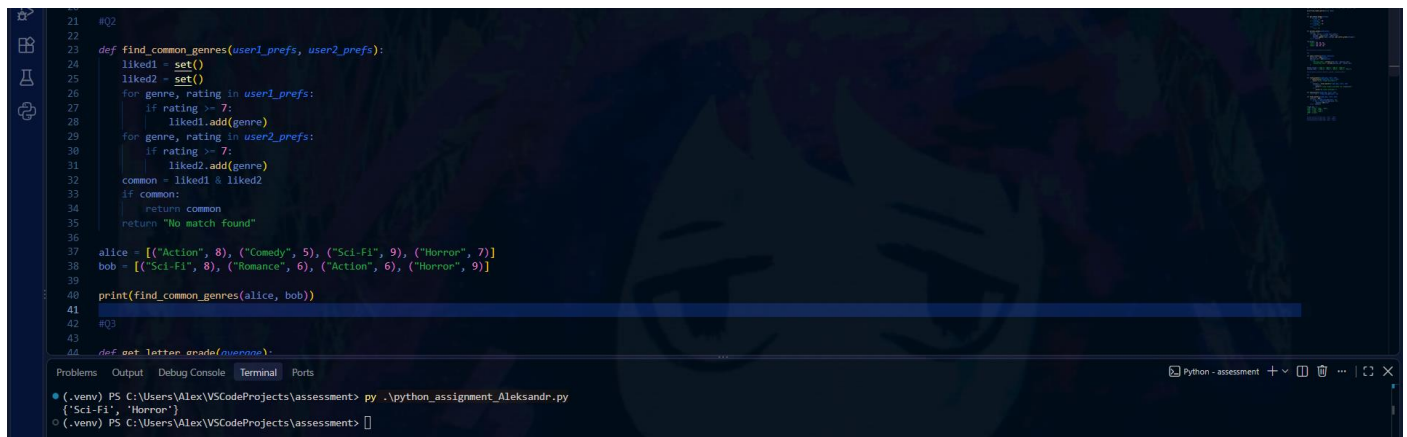
```
def find_common_genres(user1_prefs, user2_prefs):
    liked1 = set()
    liked2 = set()
    for genre, rating in user1_prefs:
        if rating >= 7:
            liked1.add(genre)
    for genre, rating in user2_prefs:
        if rating >= 7:
            liked2.add(genre)
    common = liked1 & liked2
    if common:
        return common
    return "No match found"
```

```
alice = [("Action", 8), ("Comedy", 5), ("Sci-Fi", 9), ("Horror", 7)]
```

```
bob = [("Sci-Fi", 8), ("Romance", 6), ("Action", 6), ("Horror", 9)]
```

```
print(find_common_genres(alice, bob))
```

Output:



The screenshot shows a Python IDE with a dark theme. The editor displays the code from the previous blocks. The output console at the bottom shows the result of the function call: `{'Sci-Fi', 'Horror'}`. The terminal window at the bottom shows the command `py .\python_assignment_Aleksandr.py` being executed in a virtual environment.

### Question 3: Modular Student Grade Processor

The Scenario: A teacher has provided you with a dictionary where the keys are student names and the values are lists of their raw test scores. You need to calculate their average score and assign a letter grade.

Your Task (Requires Modular Functions):

1. Write a helper function called `get_letter_grade(average)`.
2. Write a main function called `process_grades(grade_book)`.
3. Loop through the dictionary keys and values.
4. Calculate the average of the list of numbers for each student.
5. Call your `get_letter_grade()` function to get their letter grade.
6. Return a List of Tuples, where each tuple looks like: (Student\_Name, Average\_Score, Letter\_Grade).

Code:

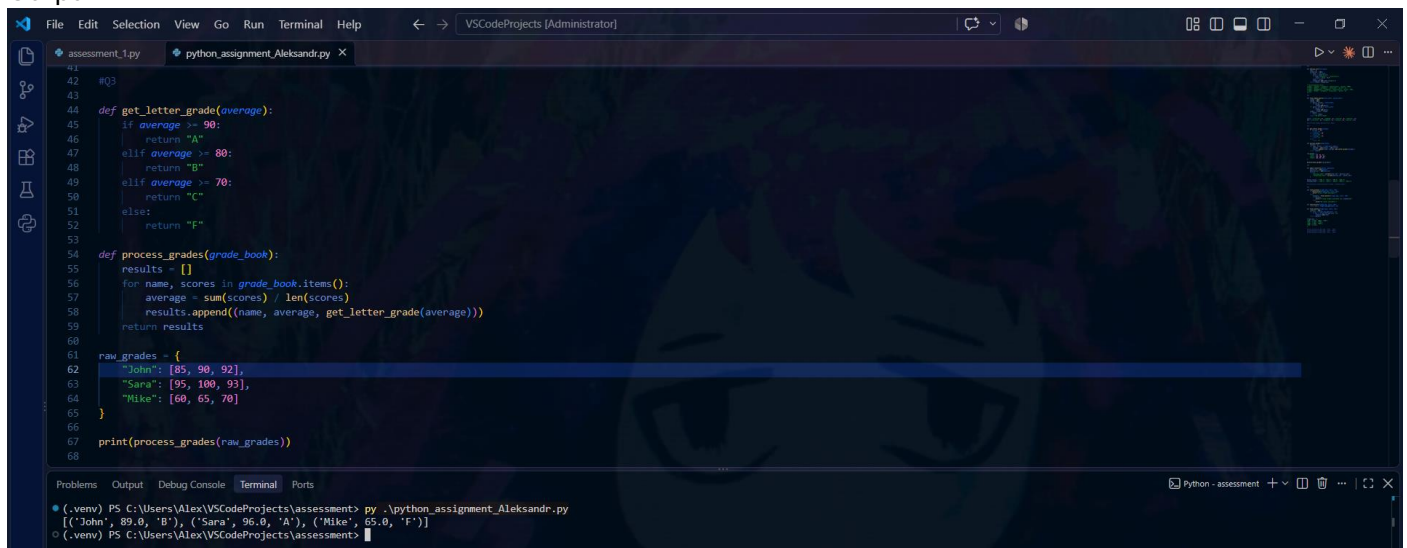
```
def get_letter_grade(average):
    if average >= 90:
        return "A"
    elif average >= 80:
        return "B"
    elif average >= 70:
        return "C"
    else:
        return "F"

def process_grades(grade_book):
    results = []
    for name, scores in grade_book.items():
        average = sum(scores) / len(scores)
        results.append((name, average, get_letter_grade(average)))
    return results
```

```
raw_grades = {"John": [85, 90, 92], "Sara": [95, 100, 93], "Mike": [60, 65, 70]}
```

```
print(process_grades(raw_grades))
```

Output:



The screenshot shows a VS Code editor window with two files open: `assessment_1.py` and `python_assignment_Aleksandr.py`. The code in `python_assignment_Aleksandr.py` is as follows:

```
41
42 #Q3
43
44 def get_letter_grade(average):
45     if average >= 90:
46         return "A"
47     elif average >= 80:
48         return "B"
49     elif average >= 70:
50         return "C"
51     else:
52         return "F"
53
54 def process_grades(grade_book):
55     results = []
56     for name, scores in grade_book.items():
57         average = sum(scores) / len(scores)
58         results.append((name, average, get_letter_grade(average)))
59     return results
60
61 raw_grades = {
62     "John": [85, 90, 92],
63     "Sara": [95, 100, 93],
64     "Mike": [60, 65, 70]
65 }
66
67 print(process_grades(raw_grades))
68
```

The terminal output at the bottom shows the execution of the code:

```
(.venv) PS C:\Users\Alex\VSCodeProjects\assessment> py .\python_assignment_Aleksandr.py
[('John', 86.0, 'B'), ('Sara', 96.0, 'A'), ('Mike', 65.0, 'C')]
(.venv) PS C:\Users\Alex\VSCodeProjects\assessment>
```

#### Question 4: The Inventory Discrepancy Finder

The Scenario: You are doing an inventory audit. You have a `master_list` of items that should be in the warehouse, and a `physical_count` list of items actually found on the shelves. Sometimes employees scan the same item twice, so the physical count might have duplicates.

Your Task:

1. Write a function called `audit_inventory(master, physical)`.
2. Strip any duplicates from the `physical_count`.
3. Identify which items are missing (in the master list, but not found physically).
4. Identify which items are unexpected (found physically, but not on the master list).
5. Return a Dictionary containing two keys: "missing\_items" and "unexpected\_items", with their corresponding values being lists of those items.

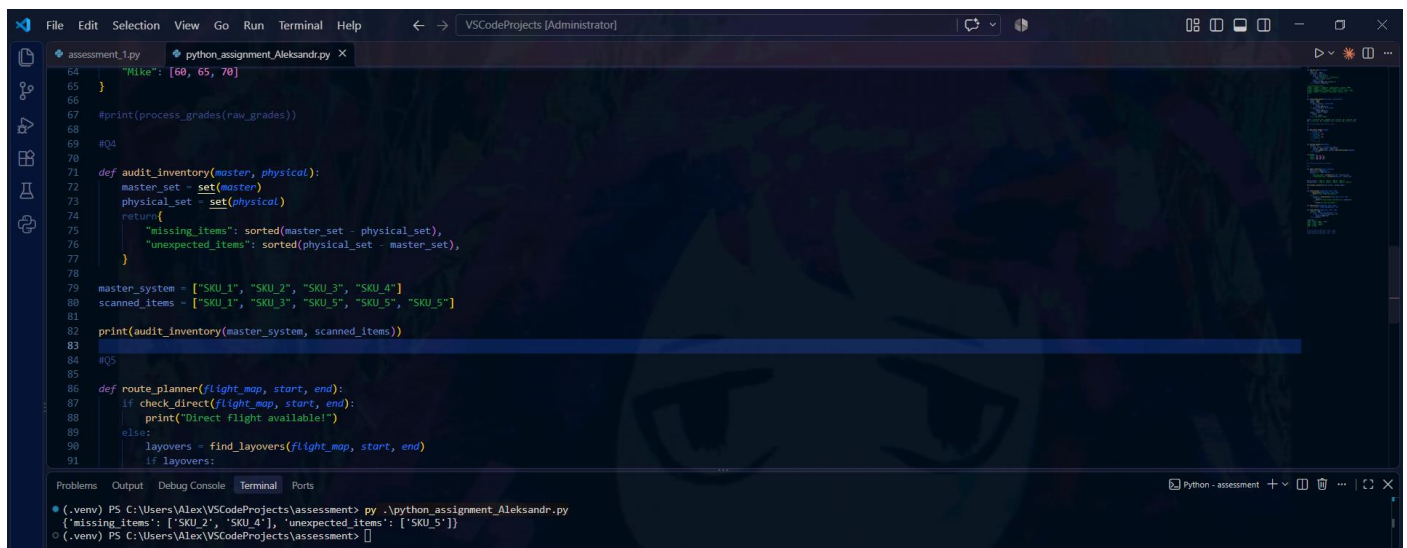
Code:

```
def audit_inventory(master, physical):
    master_set = set(master)
    physical_set = set(physical)
    return{
        "missing_items": sorted(master_set - physical_set),
        "unexpected_items": sorted(physical_set - master_set),
    }
```

```
master_system = ["SKU_1", "SKU_2", "SKU_3", "SKU_4"]
scanned_items = ["SKU_1", "SKU_3", "SKU_5", "SKU_5", "SKU_5"]

print(audit_inventory(master_system, scanned_items))
```

Output:



The screenshot shows a VS Code editor window with a Python file named `python_assignment_Aleksandr.py`. The code defines the `audit_inventory` function and uses it with `master_system` and `scanned_items`. The terminal output shows the function's return value: `{'missing_items': ['SKU_2', 'SKU_4'], 'unexpected_items': ['SKU_5']}`.

```
File Edit Selection View Go Run Terminal Help
python_assignment_Aleksandr.py
64 "Mike": [60, 65, 70]
65 }
66
67 #print(process_grades(raw_grades))
68
69 #Q4
70
71 def audit_inventory(master, physical):
72     master_set = set(master)
73     physical_set = set(physical)
74     return{
75         "missing_items": sorted(master_set - physical_set),
76         "unexpected_items": sorted(physical_set - master_set),
77     }
78
79 master_system = ["SKU_1", "SKU_2", "SKU_3", "SKU_4"]
80 scanned_items = ["SKU_1", "SKU_3", "SKU_5", "SKU_5", "SKU_5"]
81
82 print(audit_inventory(master_system, scanned_items))
83
84 #Q5
85
86 def route_planner(flight_map, start, end):
87     if check_direct(flight_map, start, end):
88         print("Direct flight available!")
89     else:
90         layovers = find_layovers(flight_map, start, end)
91         if layovers:
```

```
Python - assessment
(.venv) PS C:\Users\Alex\VSCodeProjects\assessment> py .\python_assignment_Aleksandr.py
{'missing_items': ['SKU_2', 'SKU_4'], 'unexpected_items': ['SKU_5']}
```

### Question 5: The Flight Routing System

The Scenario: You are building a basic flight pathfinder. You have a dictionary where the keys are departure cities, and the values are lists of destination cities you can fly directly to. You need to find if someone can fly directly, or if they need a 1-stop layover.

Your Task (Requires Modular Functions):

1. Write a helper function `check_direct(flights_dict, start, end)` that returns True if end is in the start city's list of destinations, and False otherwise.
2. Write a helper function `find_layovers(flights_dict, start, end)` that loops through the destinations of the start city. If any of those cities have a flight to the end city, add that layover city to a Set. Return the Set.
3. Write a main function `route_planner(flights_dict, start, end)` that calls both helpers.
4. It should print "Direct flight available!" if true.
5. If no direct flight exists, but layovers do, print "1-stop flights available via: [layover cities]".
6. If neither exist, print "No routes available."

Code:

```
def route_planner(flight_map, start, end):
    if check_direct(flight_map, start, end):
        print("Direct flight available!")
    else:
        layovers = find_layovers(flight_map, start, end)
        if layovers:
            print(f"1-stop flights available via: {layovers}")
        else:
            print("No routes available.")
```

```
def check_direct(flight_map, start, end):
    return end in flight_map.get(start, [])
```

```
def find_layovers(flight_map, start, end):
    layovers = set()
    for city in flight_map.get(start, []):
        if end in flight_map.get(city, []):
            layovers.add(city)
    return layovers
```

```
flight_map = {
    "JFK": ["LAX", "MIA", "LHR"],
    "MIA": ["LAX", "DFW"],
    "LHR": ["CDG", "DXB"],
    "DFW": ["LAX"]
}
```

```
route_planner(flight_map, "JFK", "LHR")
route_planner(flight_map, "JFK", "DFW")
route_planner(flight_map, "MIA", "CDG")
```

Output:

File Edit Selection View Go Run Terminal Help

VSCodeProjects [Administrator]

assessment\_1.py python\_assignment\_Aleksandr.py

```
84 #Q5
85
86 def route_planner(flight_map, start, end):
87     if check_direct(flight_map, start, end):
88         print("Direct flight available!")
89     else:
90         layovers = find_layovers(flight_map, start, end)
91         if layovers:
92             print(f"1-stop flights available via: {layovers}")
93         else:
94             print("No routes available.")
95
96 def check_direct(flight_map, start, end):
97     return end in flight_map.get(start, [])
98
99 def find_layovers(flight_map, start, end):
100     layovers = set()
101     for city in flight_map.get(start, []):
102         if end in flight_map.get(city, []):
103             layovers.add(city)
104     return layovers
105
106 flight_map = {
107     "JFK": ["LAX", "MIA", "LHR"],
108     "MIA": ["LAX", "DFW"],
109     "LHR": ["CDG", "DXB"],
110     "DFW": ["LAX"]
111 }
112
113 route_planner(flight_map, "JFK", "LHR")
114 route_planner(flight_map, "JFK", "DFW")
115 route_planner(flight_map, "MIA", "CDG")
```

Problems Output Debug Console Terminal Ports

Python - assessment

```
(.venv) PS C:\Users\Alex\VSCodeProjects\assessment> py .\python_assignment_Aleksandr.py
Direct flight available!
1-stop flights available via: {'MIA'}
No routes available.
(.venv) PS C:\Users\Alex\VSCodeProjects\assessment> 
```